

FIREFIGHTING DRONE SWARM (RTCS – SIM)

SYSC3303 – A3G6 • Winter 2026 • Dr. Sabouni, Rami • Carleton University (FED – SCE)

Table of Contents

- [1. Overview](#)
- [2. Features](#)
- [3. System Architecture](#)
 - [3.1 Components and Connectors \(C&C\)](#)
 - [3.2 Main Files](#)
 - [3.2.1 Entry Points](#)
 - [3.2.2 Core Subsystem Files](#)
 - [3.2.3 GUI Files](#)
 - [3.2.4 Models & Messages](#)
 - [3.2.5 UDP Support](#)
 - [3.3 Input Files](#)
 - [3.3.1 Updated CSV Format](#)
 - [3.3.2 Column Template](#)
 - [3.3.3 Example CSV](#)
 - [3.3.4 Allowed Values](#)
 - [3.4 Diagrams](#)
 - [3.4.1 UML Class Diagram](#)
 - [3.4.2 Sequence Diagrams](#)
 - [3.4.3 State Machine Diagrams](#)
 - [3.4.4 Timing Diagrams](#)
 - [3.5 Design Summary](#)
 - [3.5.1 Iteration 02](#)
 - [3.5.3 Iteration 03](#)
 - [3.5.4 Iteration 04](#)
 - [3.5.5 Iteration 05](#)
- [4. Installation & Setup](#)
 - [4.1 Requirements](#)

- [4.2 Getting the Project](#)
- [4.3 Opening in IntelliJ](#)
- [4.4 Building the Project](#)
- [4.5 Running the Simulation](#)
- [4.6 Input Files](#)
 - [4.6.1 Event File \(CSV\)](#)
 - [4.6.2 Zone File \(CSV\)](#)
- [4.7 Configuration](#)
- [5. System Behaviour](#)
 - [5.1 Fire Incident Subsystem](#)
 - [5.2 Scheduler](#)
 - [5.3 Drone Subsystem](#)
- [6. Testing and V&V](#)
 - [6.1 Unit Tests](#)
 - [6.2 Integration Tests](#)
 - [6.3 System Tests](#)
 - [6.4 Measurement Results](#)
- [7. Team Contributions](#)
- [8. Reflection](#)

1. OVERVIEW

This Real-Time Control System (RTCS) simulates a firefighting drone swarm composed of three logical subsystems:

1. Scheduler
2. Drone Subsystem
3. Fire Incident Subsystem

The current codebase runs the Scheduler, Fire Incident subsystem, and Drone subsystem as separate UDP-connected Java processes. Drones register independently with the Scheduler, which dispatches work and keeps the runtime GUI updated.

Team Members

- Pietro Adamvoski (101238885)
- Avery Robertson (101288279)

- Adam Haddadin (101308378)
- Fareen Lavji (xxxxxx543)

2. Features

- ☒ Iteration 1: Basic message passing between subsystems using threaded communication.
- ☒ Iteration 2: Core scheduling logic and complete drone state machine implementation.
- ☒ Iteration 3: Multi-drone support with distributed execution over UDP.
- ☒ Iteration 4: Fault injection, detection, and automated fault handling.
- ☒ Iteration 5: Full real-time visualization, capacity limits, and system performance metrics.

3. System Architecture

3.1 Components and Connectors (C&C)

The current Iteration 5 source tree is organized under `src/main/java/DroneSwarmSim`, where each subsystem is isolated into its own top-level package and communicates over UDP:

```
DroneSwarmSim/      # Root package

config/             # Runtime configuration classes
├── DroneConfig.java
├── FireIncidentConfig.java
└── SchedulerConfig.java

drone/              # Drone Subsystem & drone state logic
├── DroneInfo.java
├── DroneMain.java
├── DroneState.java
├── DroneStatusUpdate.java
├── DroneSubsystem.java
└── DroneTask.java

fire/               # Fire Incident Subsystem
├── FireIncident.java
├── FireIncidentMain.java
├── FireIncidentSubsystem.java
└── EventType.java

messaging/          # Shared UDP message DTOs
├── Ack.java
├── AssignTask.java
├── DroneRegister.java
├── DroneUpdate.java
├── FaultReport.java
├── IncidentReport.java
└── MessageType.java

model/              # Shared domain models & utilities
├── Severity.java
└── Timer.java
```

```

└── Zone.java

net/udp/                # UDP transport and packet serialization
├── PacketBuilder.java
├── PacketParser.java
├── UdpEndpoint.java
├── UdpReceiver.java
└── UdpSender.java

scheduler/              # Scheduler core + scheduling logic
├── FaultTypes.java
├── Scheduler.java
├── SchedulerMain.java
├── SchedulingState.java
├── SimulationLogger.java
└── SimulationMetrics.java

ui/                      # User Interface components
├── GUI.java
├── RuntimeGUI.java
└── TileTypes.java

```

3.2. Main files

3.2.1 Entry points

- DroneSwarmSim.scheduler.SchedulerMain
- DroneSwarmSim.fire.FireIncidentMain
- DroneSwarmSim.drone.DroneMain

3.2.2. Core subsystem files

- Scheduler.java
- FaultTypes.java
- FireIncidentSubsystem.java
- DroneSubsystem.java

3.2.3. GUI files

- GUI.java
- RuntimeGUI.java
- TileTypes.java

3.2.4. Models & Messages

- Zone.java, Severity.java, Timer.java
- FaultReport.java

3.2.5. UDP Support

- `UdpEndpoint.java`, `UdpSender.java`, `UdpReceiver.java`
- `PacketBuilder.java`, `PacketParser.java`

3.3. Input Files

3.3.1. Updated CSV Format

Iteration 4 added a `FaultType` column so fault scenarios can be scripted directly from the event file.

3.3.2. Column Template

Time, ZoneId, EventType, Severity, FaultType

3.3.3. Example CSV

```
Time,ZoneId,EventType,Severity,FaultType
14:03:15,3,FIRE_DETECTED,High,NOZZLE_STUCK_OPEN
14:07:00,1,FIRE_DETECTED,Moderate,NONE
14:09:22,2,DRONE_REQUEST,Low,DRONE_STUCK
14:10:00,1,DRONE_REQUEST,High,NOZZLE_STUCK_CLOSED
14:12:30,4,FIRE_DETECTED,Low,NONE
```

3.3.4. Allowed Values

- EventType: FIRE_DETECTED, DRONE_REQUEST
- Severity: Low, Moderate, High
- FaultType: NONE, DRONE_STUCK, NOZZLE_STUCK_CLOSED, NOZZLE_STUCK_OPEN, PACKET_LOSS, PACKET_CORRUPTION

3.4 Diagrams

3.4.1. UML Class Diagram

```
classDiagram
    direction LR

    %% ENTRY POINTS
    class SchedulerMain { +main(args) }
    class FireIncidentMain { +main(args) }
    class DroneMain { +main(args) }

    %% CORE SUBSYSTEMS
    class Scheduler {
        +run()
        +getZones()
        +processIncident(IncidentReport)
        +registerDrone(DroneRegister)
        +updateDrone(DroneUpdate)
        +handleFault(droneId, faultType)
        +reassign(zoneId)
```

```

}

class FireIncidentSubsystem {
    +run()
    +parseEventFile(path)
    +dispatchIncident(IncidentReport)
}

class DroneSubsystem {
    +run()
    +injectFaultIfAny()
    +applyFault(faultType)
    +moveTowardTarget()
    +simulateDrop()
    +sendTelemetry()
}

%% DATA + TRACKING
class DroneInfo {
    +getState()
    +faultState
    +updateTelemetry(state, xPos, yPos, water, battery, fuel)
    +assignZone(zoneId)
    +clearAssignedZone()
    +incrementCompletedAssignments()
}

%% GUI
class GUI {
    +start()
    +showFireIncident(zoneId, severityLabel)
    +showAssignment(droneId, zoneId, severityLabel, remainingWater, status)
    +updateAssignmentProgress(droneId, zoneId, severityLabel, remainingWater,
status)
    +showDronePosition(droneId, x, y)
    +clearFireIncident(zoneId)
    +clearAssignment(droneId)
    +showFault(droneId, faultType)
    +showDroneOffline(droneId)
    +updateDroneBatteryFuel(droneId, batteryPct, fuelPct)
    +logEvent(message)
    +logTelemetry(message)
    +logQueueEvent(message)
    +logAssignmentHistory(message)
}

class RuntimeGUI

%% NETWORKING (UDP)
class UdpSender { +send(data) }
class UdpReceiver {
    +start()
    +close()
}

class PacketBuilder {
    +build(IncidentReport)
    +build(AssignTask)
    +build(DroneRegister)
}

```

```

        +build(DroneUpdate)
        +build(FaultReport)
    }

class PacketParser {
    +getType(data)
    +parseIncidentReport(data)
    +parseAssignTask(data)
    +parseDroneRegister(data)
    +parseDroneUpdate(data)
    +parseFaultReport(data)
}

%% MESSAGES / EVENTS
class IncidentReport {
    +time
    +zoneId
    +eventType
    +severity
    +faultType
}

class AssignTask {
    +droneId
    +zoneId
    +zoneXOrigin
    +zoneYOrigin
    +zoneWidth
    +zoneLength
    +severity
}

class DroneRegister {
    +droneId
    +host
    +port
}

class DroneUpdate {
    +droneId
    +state
    +xPos
    +yPos
    +water
    +battery
    +fuel
}

class FaultReport {
    +droneId
    +faultType
}

%% STATIC MODELS
class Zone {
    +zoneID
    +xOrigin
    +yOrigin
    +xEnd

```

```

        +yEnd
        +centerX
        +centerY
    }

    class Severity { <<enumeration>> }
    class EventType { <<enumeration>> }
    class DroneState { <<enumeration>> }
    class SchedulingState { <<enumeration>> }
    class FaultTypes {
        <<enumeration>>
        NONE
        DRONE_STUCK
        NOZZLE_STUCK_CLOSED
        NOZZLE_STUCK_OPEN
        PACKET_LOSS
        PACKET_CORRUPTION
    }

%% RELATIONSHIPS
SchedulerMain --> Scheduler
SchedulerMain --> RuntimeGUI
RuntimeGUI --|> GUI

FireIncidentMain --> FireIncidentSubsystem
DroneMain --> DroneSubsystem

Scheduler --> Zone
Scheduler --> IncidentReport
Scheduler --> AssignTask
Scheduler --> DroneInfo
Scheduler --> GUI
Scheduler --> UdpReceiver
Scheduler --> UdpSender
Scheduler --> PacketParser
Scheduler --> PacketBuilder
Scheduler --> FaultTypes

FireIncidentSubsystem --> IncidentReport
FireIncidentSubsystem --> Severity
FireIncidentSubsystem --> EventType
FireIncidentSubsystem --> UdpSender
FireIncidentSubsystem --> PacketBuilder

DroneSubsystem --> AssignTask
DroneSubsystem --> DroneRegister
DroneSubsystem --> DroneUpdate
DroneSubsystem --> FaultReport
DroneSubsystem --> UdpReceiver
DroneSubsystem --> UdpSender
DroneSubsystem --> PacketParser
DroneSubsystem --> PacketBuilder
DroneSubsystem --> DroneState
DroneSubsystem --> FaultTypes

IncidentReport --> EventType
IncidentReport --> Severity
IncidentReport --> FaultTypes

```

```
AssignTask --> Severity
DroneUpdate --> DroneState
FaultReport --> FaultTypes

GUI --> Zone : renders
```

3.4.2. Sequence Diagrams

3.4.2.1. Messages Sequence Diagram

```
sequenceDiagram
    autonumber

    participant FIS as FireIncidentSubsystem
    participant SCH as Scheduler
    participant GUI as RuntimeGUI
    participant D0 as Drone0
    participant D1 as Drone1

    FIS->>SCH: IncidentReport over UDP
    SCH->>GUI: showFireIncident(zoneId, severity)
    D0->>SCH: DroneRegister(droneId=0, port=7000)
    D1->>SCH: DroneRegister(droneId=1, port=7001)

    SCH->>SCH: queue incident and choose best idle drone
    SCH->>GUI: showAssignment(droneId, zoneId, severity, remainingWater, status)
    SCH->>D0: AssignTask over UDP

    D0->>SCH: DroneUpdate(EN_ROUTE, xPos, yPos, water, battery, fuel)
    SCH->>GUI: showDronePosition(droneId, xPos, yPos)
    SCH->>GUI: updateDroneBatteryFuel(droneId, battery, fuel)
    SCH->>GUI: updateAssignmentProgress(...)

    D0->>SCH: DroneUpdate(ARRIVED, xPos, yPos, water, battery, fuel)
    SCH->>GUI: updateAssignmentProgress(...)

    D0->>SCH: DroneUpdate(DROPPING_AGENT, xPos, yPos, water, battery, fuel)
    SCH->>GUI: updateAssignmentProgress(...)

    D0->>SCH: DroneUpdate(COMPLETED, xPos, yPos, waterAfterDrop, battery, fuel)
    alt zone still needs water
        SCH->>GUI: logAssignmentHistory(partial drop)
        SCH->>GUI: logQueueEvent(re-queueing zone)
        SCH->>SCH: put incident back into queue
        SCH->>D1: AssignTask over UDP
    else zone extinguished
        SCH->>GUI: clearFireIncident(zoneId)
        SCH->>GUI: logAssignmentHistory(zone extinguished)
    end

    D0->>SCH: DroneUpdate(RETURNING, xPos, yPos, water, battery, fuel)
    SCH->>GUI: updateAssignmentProgress(...)

    D0->>SCH: DroneUpdate(IDLE, 0, 0, 15.0, 100, 100)
    SCH->>GUI: clearAssignment(droneId)
    SCH->>GUI: clearDronePosition(droneId)
```

3.4.2.2. Fault Handling Sequence Diagram

```
sequenceDiagram
    autonumber
    participant FIS as FireIncidentSubsystem
    participant SCH as Scheduler
    participant GUI as RuntimeGUI
    participant D0 as Drone0
    participant D1 as Drone1

    FIS->>SCH: IncidentReport(zoneId, severity)
    SCH->>GUI: showFireIncident(zoneId)
    D0-->>SCH: DroneRegister(0)
    D1-->>SCH: DroneRegister(1)

    SCH->>D0: AssignTask(zoneId)
    D0-->>SCH: DroneUpdate(EN_ROUTE, xPos, yPos, water, battery, fuel)

    alt D0 encounters injected fault
        D0-->>SCH: FaultReport(NOZZLE_STUCK_CLOSED)
        SCH->>GUI: showFault(D0, NOZZLE_STUCK_CLOSED)
        SCH->>SCH: mark D0 unavailable
        SCH->>SCH: select alternate idle drone
        SCH->>D1: AssignTask(zoneId)
        D1-->>SCH: DroneUpdate(EN_ROUTE, xPos, yPos, water, battery, fuel)
    else D0 completes mission normally
        D0-->>SCH: DroneUpdate(ARRIVED, xPos, yPos, water, battery, fuel)
        D0-->>SCH: DroneUpdate(DROPPING_AGENT, xPos, yPos, water, battery, fuel)
        D0-->>SCH: DroneUpdate(COMPLETED, xPos, yPos, water, battery, fuel)
        SCH->>GUI: clearFireIncident(zoneId)
    end
end
```

3.4.3. State Machine Diagrams

3.4.3.1. Scheduler State Machine

```
stateDiagram-v2
    [*] --> WAITING
    WAITING --> ASSIGNING: IncidentReport
    ASSIGNING --> MONITORING: AssignTask
    ASSIGNING --> WAITING: No idle drones
    MONITORING --> MONITORING: DroneUpdate
    MONITORING --> FAULT_HANDLING: FaultReport
    FAULT_HANDLING --> ASSIGNING: Soft fault
    FAULT_HANDLING --> OFFLINE: Hard fault
    OFFLINE --> WAITING
    WAITING --> WAITING: DroneRegister
```

3.4.3.2. Drone State Machine

```
stateDiagram-v2
    [*] --> IDLE
    IDLE --> EN_ROUTE
    EN_ROUTE --> ARRIVED
    EN_ROUTE --> FAULTED: DRONE_STUCK (timeout)
    ARRIVED --> DROPPING_AGENT
    DROPPING_AGENT --> COMPLETED
    DROPPING_AGENT --> FAULTED: NOZZLE_STUCK_CLOSED
    DROPPING_AGENT --> FAULTED: NOZZLE_STUCK_OPEN
```

```
COMPLETED --> RETURNING
RETURNING --> IDLE
FAULTED --> [*]
```

3.4.4. Timing Diagrams

3.4.4.1. Normal Mission Timing

```
gantt
    title Single-drone mission timeline
    dateFormat X
    axisFormat %Ls

    section Drone
    Receive assignment      :a1, 0, 1s
    Travel to zone         :a2, after a1, 6s
    Drop agent             :a3, after a2, 4s
    Return to base         :a4, after a3, 6s
    Refill / reset         :a5, after a4, 2s

    section Scheduler
    Queue and dispatch     :b1, 0, 1s
    Monitor telemetry      :b2, 1, 16s
    Mark fire extinguished :b3, 11, 1s
```

3.4.4.2. Fault and Reassignment Timing

```
gantt
    title Fault handling and reassignment timeline
    dateFormat X
    axisFormat %Ls

    section Drone 0
    Receive assignment      :c1, 0, 1s
    Travel / partial work   :c2, after c1, 4s
    Fault report or timeout :c3, after c2, 1s

    section Scheduler
    Detect fault             :d1, 5, 1s
    Re-queue incident       :d2, after d1, 1s
    Dispatch replacement    :d3, after d2, 1s

    section Drone 1
    Travel to zone          :e1, 7, 6s
    Drop agent              :e2, after e1, 4s
    Return / refill         :e3, after e2, 8s
```

3.5. Design Summary

3.5.1. Iteration 02

Iteration 2 expands the system from simple message-passing to coordinated control:

- The Scheduler uses a state machine to receive incidents, assign tasks to drones, and handle faults.
- The Drone Subsystem executes missions through its own state machine, managing navigation,

water dropping, refilling, and completion reporting.

- The Fire Incident Subsystem streams timed events into the system and receives acknowledgements after tasks are completed.
- Communication remains decoupled through dedicated channel classes, ensuring each subsystem can be tested in isolation.
- This architecture provides the groundwork required for multi-drone coordination, distributed execution, and error handling in future iterations.

3.5.3. Iteration 03

All inter-subsystem communication now occurs over UDP. The old thread-channel architecture from Iterations 1–2 has been replaced by packet-based communication between standalone main classes.

3.5.4. Iteration 04

- Fault injection through the event CSV fault column
- Fault state machine for drones (soft and hard faults)
- Travel timeout detection (DRONE_STUCK)
- Nozzle jam detection (NOZZLE_STUCK_CLOSED, NOZZLE_STUCK_OPEN)
- Hard-fault drone shutdown (SHUTDOWN)
- Scheduler-level reassignment
- GUI overlays for faults and offline drones
- Updated message types for error propagation

3.5.5. Iteration 05

- The runtime GUI grew into a proper monitoring dashboard. It now shows the live grid, fire state, fault overlays, per-drone selection, fleet status, elapsed time, and logs you can actually scroll through without being snapped back to the bottom.
- Telemetry now includes water, battery, and fuel, with assignment history and metrics shown in both the GUI and the scheduler logs.
- Drone capacity and refill behavior are enforced, and the scheduler tracks the numbers we care about for the demo: response time, completion time, queue depth, utilization, and total fleet distance.
- The demo setup is also simpler now. The `run-demo.sh` launcher prompts for the number of drones to start, defaults to 20 if left blank, and still supports manual single-drone startup through `DroneMain` when controlled testing is needed.

4. Installation & Setup

4.1. Requirements

- ☒ Java 17 or later
- ☐ IntelliJ IDEA (recommended)
- ☐ Git (optional, for pulling updates)

4.2. Getting the Project

Clone or download the repository:

```
git clone <your-repo-url>  
cd <project-folder>
```

Or download the ZIP and extract it.

4.3. Opening in IntelliJ

1. Open IntelliJ IDEA
2. Select File → Open
3. Choose the project root folder
4. Allow IntelliJ to index and load dependencies

4.4. Building the Project

If using IntelliJ, the project builds automatically. Using the terminal (Maven):

```
mvn clean compile
```

4.5. Running the Simulation

1. The easiest way to run the demo is `./run-demo.sh` from the repository root. It compiles the project, starts the Scheduler, prompts for how many drones to launch, starts the Fire Incident subsystem, and cleans up stale background processes when you stop it. If you leave the prompt blank, it defaults to 20 drones.
2. Manual startup order:
 - Start `DroneSwarmSim.scheduler.SchedulerMain` to launch the Scheduler and runtime GUI.
 - Start `DroneSwarmSim.drone.DroneMain`.
 - Start `DroneSwarmSim.fire.FireIncidentMain` to stream incidents from the configured CSV file.
3. `DroneSwarmSim.drone.DroneMain` behavior:

- With no args, it starts the default fleet of 20 drones.
- With `-Dexec.args="<droneId>"`, it starts only that drone.

4. `run-demo.sh` launcher behavior:

- If you enter 5, it launches drones 0 through 4.
- If you enter 1, it launches only drone 0.
- If you leave the prompt blank, it launches the default fleet of 20 drones.

5. Example manual commands:

```
mvn exec:java -Dexec.mainClass="DroneSwarmSim.scheduler.SchedulerMain"
mvn exec:java -Dexec.mainClass="DroneSwarmSim.drone.DroneMain"
mvn exec:java -Dexec.mainClass="DroneSwarmSim.drone.DroneMain" -Dexec.args="3"
mvn exec:java -Dexec.mainClass="DroneSwarmSim.fire.FireIncidentMain"
```

1. Optional: pass a custom zone CSV path to `DroneSwarmSim.scheduler.SchedulerMain` or a custom event CSV path to `DroneSwarmSim.fire.FireIncidentMain`.

4.6. Input Files

The simulation uses two input files to drive events and define the environment.

4.6.1. Event File (CSV)

Each line represents a time-stamped event: `hh:mm:ss.mmm, zoneId, FIRE_DETECTED | DRONE_REQUEST, Low | Moderate | High, FaultType` Example: `14:00:00.300, 2, FIRE_DETECTED, Moderate, NOZZLE_STUCK_CLOSED`

4.6.2. Zone File (CSV)

Each zone is defined by its start and end coordinates: `zoneId, (x1;y1), (x2;y2)` Example: `3, (1800;0), (2700;900)` Severity values map to the amount of water required.

4.7. Configuration

Most simulation parameters still live in code constants or the input files:

- Default fleet size (`DroneConfig.NUM_DRONES`, currently 20)
- Number of zones
- Travel time between zones
- Agent drop time
- Nozzle open/close time
- Initial agent capacity per drone

- Battery drain and fuel usage constants
- Paths to event and zone input files
- UDP ports per subsystem
- Per-drone task/status sockets
- Fault injection options

5. System Behaviour

The system follows a modular, message-driven model in which each subsystem executes independently.

5.1. Fire Incident Subsystem

1. Reads and parses fire events from the input file
2. Publishes incidents to the Scheduler over the report channel
3. Receives acknowledgements after drone completion

5.2. Scheduler

1. Maintains the incident queue
2. Assigns tasks to available drones following scheduling rules
3. Forwards incidents and receives drone updates
4. Handles faults (Iteration 2 foundational logic in place)

5.3. Drone Subsystem

1. Executes a state machine: Idle → En Route → Arrived → Dropping Agent → Returning → Refilling → Idle
2. Reports status, resource usage, and completion back to the Scheduler
3. Applies water, battery, and fuel consumption rules during travel and drop operations

6. Testing and V&V

Automated verification is performed with JUnit 5 via `mvn test`. The current suite contains **98 tests across 23 classes** and passes with **0 failures** and **0 skipped** tests.

6.1. Unit Tests

- The unit suite covers the main code paths in the current implementation:
- Scheduler logic: `SchedulerQueueOrderTest`, `SchedulerLoadBalancingTest`, `SchedulerSelectionTest`, `SchedulerTest`, `FaultHandlingTest`,

ZoneCsvParserTest

- Drone logic: DroneStateMachineTest, DroneCapacityRefillTest, DroneFaultRecoveryTest, DroneStatusMetricsTest
- Fire incident handling: FireIncidentSubsystemTest
- UDP / message utilities: PacketRoundTripTest, PacketParserTest, MessageContractsIT, UdpLoopbackIT
- Shared model / DTO / GUI support: SeverityMappingTest, TimerTest, IncidentReportDtoTest, GUITest

6.2. Integration Tests

Integration tests verify end-to-end behaviour across the major subsystems, including:

- mission lifecycle from incident arrival to drone completion (MissionLifecycleTest)
- multi-incident queue draining and balanced load distribution (MultiIncidentQueueDrainIT)
- two-drone end-to-end mission flow (TwoDronesBasicFlowIT)
- fault injection, timeout handling, and reassignment (FaultHandlingTest)
- queued-task processing while a drone is busy (DroneStatusMetricsTest.droneQueuesMultipleTasks)
- least-loaded drone selection (SchedulerLoadBalancingTest.testLoadBalancingPrefersLeastLoadedDrone)
- path-through redirection (SchedulerLoadBalancingTest.testPathThroughRedirect)

6.3. Manual / System Validation

Display-dependent GUI/system scenarios are validated manually. Runtime logs and metrics are written to the `logs/` directory, and `Scheduler.printMetricsSummary()` writes the final metrics summary to stdout and to `logs/metrics.log`.

Note: Further V&V information is summarized in the [iteration05_vvReport.pdf](#) document.

6.4. Measurement Results

The system records runtime measurements through `SimulationMetrics`, and the summaries are written to `logs/metrics.log`. The values below were taken from a full end-to-end demo run using the default final input files:

- Event file: `src/Final_event_file_w26.csv`
- Zone file: `src/Final_zone_file_w26.csv`
- Fleet size: 20 drones launched through the default `run-demo.sh` flow

These numbers are not meant to be treated as hard real-time guarantees. They are measured results from the current implementation running on a single machine over `localhost`.

Scenario	Source	Key Results
Full 20-drone run using the final event and zone files	<code>logs/metrics.log</code> summary at <code>15:51:49.945</code>	Total duration <code>591.24 s</code> ; fires reported/extinguished <code>22 / 22</code> ; service requests <code>28</code> ; total drone trips <code>82</code> ; total faults <code>0</code> ; average response time <code>6.74 s</code> ; max response time <code>12.74 s</code> ; average completion time <code>19.07 s</code> ; max completion time <code>29.61 s</code> ; average queue length <code>0.50</code> ; max queue length <code>1</code> ; total fleet distance <code>291910.7</code> units.

For that full run, the per-fire breakdown in `logs/metrics.log` recorded these arrival and extinguish times:

Arrival time means the time from when an incident is reported to when the first assigned drone reaches that zone. Extinguish time means the time from when the incident is reported to when the fire is fully out. For low-severity fires, those numbers are usually fairly close. For moderate and high fires, the gap is larger because the total includes travel, drop time, and sometimes a follow-up drone if one drop is not enough.

Zone	Arrival Time	Extinguish Time
2 @ 00:03:32	6.81 s	10.81 s
3 @ 00:16:16	5.32 s	18.64 s
4 @ 00:17:21	10.70 s	26.87 s
5 @ 01:24:54	8.88 s	12.88 s
4 @ 01:48:35	8.18 s	28.65 s
5 @ 01:57:35	12.74 s	29.61 s
2 @ 01:59:24	5.52 s	20.29 s
2 @ 02:30:12	6.77 s	10.77 s
3 @ 02:31:45	5.31 s	18.63 s
5 @ 03:05:49	8.88 s	25.75 s

Zone	Arrival Time	Extinguish Time
3 @ 03:50:25	5.31 s	18.63 s
1 @ 03:50:48	2.50 s	6.50 s
2 @ 04:05:19	6.77 s	21.54 s
4 @ 04:57:36	8.17 s	24.34 s
3 @ 04:57:50	5.53 s	18.85 s
2 @ 05:06:30	6.77 s	10.77 s
1 @ 05:21:49	2.46 s	12.91 s
4 @ 05:54:55	8.17 s	24.34 s
1 @ 06:21:16	2.46 s	12.92 s
3 @ 06:23:01	5.32 s	18.63 s
5 @ 06:44:30	8.87 s	25.75 s
2 @ 06:54:24	6.77 s	21.53 s

These numbers match what we saw during the full run. Low-severity incidents are generally cleared faster, while the longer completion times come from the larger fires that need more travel time or more than one drop. The queue measurements also stayed low across the run, which is what we wanted from the scheduler.

Refer to the [V&V Report](#) for more details.

7. Team Contributions

Team Member	Iteration 1–2	Iteration 3	Iteration 4	Iteration 5
Pietro Adamvoski (101238885)	Fire Incident and Scheduler Subsystem.	Setup workflow, documentation, diagrams	Fault handling integration in the scheduler, Iteration 4 setup notes, and timing-diagram cleanup.	Final GUI usability fixes, demo workflow cleanup, default 20-drone launch, merge/rebase conflict resolution, and final integration pass after syncing with <code>main</code> .
Avery Robertson (101288279)	GUI, Drone Subsystem, communication between threads, and Iteration 02	Assignment/ Status UI, map + log views	GUI fault overlays, offline indicators, logs	Runtime GUI dashboard polish, live visualization support, per-drone status display, and

Team Member	Iteration 1–2	Iteration 3	Iteration 4	Iteration 5
	UML and state machine diagrams.			grid rendering updates for telemetry, fire state, and assignments.
Adam Haddadin (101308378)	Iteration 01 UMLs, README.txt, initial test suites, and bug fixes.	UDP integration, test cleanup/ updates	Fault-handling test updates, packet parsing support, and verification pass on reassignment flows.	Timing diagrams, verification support, and final review of scheduler/drone behavior under the Iteration 5 visualization and metrics requirements.
Fareen Lavji (xxxxxx543)	V&V requirements + test suite packages + source code packages for Iteration 03, Iteration 02 README.md, and Iteration 03 UML and state machine diagrams.	Architecture Refactor + V&V Implementation, UDP upgrades/ integrations + full code sweep for javadocs/ test updates, README.md with mermaid diagrams + vvChecklist.m d + refactorPlan.md, issue(s) tracking + bugs + merge conflicts → devops CI/CD pipeline on GitHub.	iteration04_v &vPlan.md + README.md with updates and required diagrams in mermaid script, issue tracking with TA's feedback from iteration 03 + outlined requirements for iteration 04 → GitHub issues for traceability, derived fault CSV specifications + fault logic implement.	Implemented logic for telemetry, metrics, and other logs; refined scheduler; refactor and cleanup; drafted final README.md, README.pdf, iteration05_v vReport, and report; final V&V before submission.

8. Reflection

The part of the project we are proudest of is the GUI. We spent a lot of time on it. By the end, the system had moved well past a basic grid with a few status labels. The live map, assignment panel, logs, fault indicators, fleet information, and per-drone telemetry made the simulation much easier to understand while it was running. That mattered more and more once the number of drones increased and faults started happening in the middle of active missions.

One thing we did not fully implement was true redirection logic in the sense of resource-aware retasking. For example, if a drone used 10 L to extinguish a LOW fire and still had 5 L left, an ideal version of the system could send that drone to contribute the remaining 5 L to another nearby fire before returning to base. We talked about that idea a lot, but it turned out to be much harder than it looked because it touches scheduling, mission state, remaining-water tracking, and GUI synchronization all at once. Given the time we had, we decided it was better to keep that behavior conservative and stable instead of forcing in a half-working version.

If we were redoing part of the design, it would be the way mission timing and fire ownership are represented internally. Once partial drops, reassignment timing, faults, queueing, and GUI updates all started interacting, a lot of the late debugging effort came from keeping those pieces synchronized. A cleaner incident lifecycle model would have made the system easier to reason about and easier to test.

We were also never fully convinced by the way the "fastest drone" style comparison could be interpreted. In a simulation like this, raw speed can be changed by tuning constants rather than improving the underlying design. For example, we could reduce the drone movement update interval, shorten the water drop duration, shorten refill time, or compress the event playback timing. All of those changes would make the simulation look faster and improve the reported timings, even though they would not necessarily reflect a better scheduling strategy or a better system overall. That made the competition aspect feel a little unclear to us, since a team could end up optimizing constants rather than architecture.

Overall, we are happy with where the project landed by Iteration 5. The final system shows separate communicating processes, live visualization, fault handling, telemetry, and measurable performance. It is not perfect, but it is much closer to a real control-system simulation than what we had at the beginning.